3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

В данном разделе представлено описание классов, разработанных в рамках реализации программного средства. Рассматриваемые классы сгруппированы в соответствии с архитектурой системы и относятся к клиентской части мобильного приложения и серверной части. Для каждого класса приведены его назначение, основные свойства и методы, используемые при реализации функциональных возможностей программного средства. Такое представление обеспечивает отражение структуры программной реализации, взаимосвязи компонентов системы и распределения ответственности между отдельными классами.

Диаграмма классов представлена на чертеже ГУИР.400201.311 РР.1.

3.1 Структура базы данных

База данных программного средства реализована в виде реляционной структуры и предназначена для хранения информации о пользователях, профилях, волонтёрских мероприятиях, заявках на участие, уведомлениях, группах, чатах, посещаемости и рейтинговых операциях. Данные организованы в виде взаимосвязанных таблиц, каждая из которых отражает отдельную сущность предметной области. Для хранения и обработки информации используется PostgreSQL, а взаимодействие серверной части приложения с базой данных осуществляется посредством ORM SQLAlchemy.

3.1.1 Таблица пользователей

Таблица `users` хранит сведения об учётных записях пользователей и используется для идентификации, авторизации, определения роли и контроля доступности аккаунта.

Поля таблицы:

– `id` – уникальный идентификатор пользователя;

– `username` – уникальное имя пользователя, используемое при входе в систему;

– `password_hash` – хешированный пароль пользователя;

– `email` – адрес электронной почты пользователя;

– `email_verified` – флаг подтверждения электронной почты;

– `role` – роль пользователя, которая может принимать значения `user` или `admin`;

– `is_active` – флаг, указывающий, активна ли учётная запись;

– `is_banned` – флаг, указывающий, заблокирован ли пользователь;

– `frozen_until` – дата и время временной заморозки аккаунта;

– `created_at` – дата и время создания пользователя.

3.1.2 Таблица refresh-токенов

Таблица `refresh_tokens` используется для продления

пользовательской сессии. В базе данных хранится не исходный refresh-токен, а его хеш, что повышает безопасность системы.

Поля таблицы:

– `id` – уникальный идентификатор токена;

– `user_id` – внешний ключ на пользователя;

– `token_hash` – хеш refresh-токена;

– `expires_at` – дата истечения срока действия токена;

– `created_at` – дата создания токена;

– `revoked_at` – дата отзыва токена, если он был отозван.

3.1.3 Таблица профилей

Таблица `profiles` хранит расширенную информацию о пользователе. Профиль может относиться к волонтёру или организации, что влияет на набор отображаемых данных и доступные сценарии работы.

Поля таблицы:

– `id` – уникальный идентификатор профиля;

– `user_id` – внешний ключ на таблицу `users`;

– `type` – тип профиля: `volunteer` или `organization`;

– `avatar_url` – ссылка на загруженное изображение профиля;

– `first_name` – имя волонтёра;

– `last_name` – фамилия волонтёра;

– `organization_name` – название организации;

– `phone` – номер телефона пользователя;

– `email` – адрес электронной почты профиля;

– `city` – город пользователя;

– `country` – страна пользователя;

– `about` – дополнительная информация о пользователе или организации;

– `rating` – текущий рейтинг профиля;

– `is_verified` – флаг, указывающий, подтверждена ли организация;

– `created_at` – дата создания профиля;

– `updated_at` – дата последнего изменения профиля.

3.1.4 Таблица навыков волонтёра

Таблица `skills` хранит список навыков, которые могут быть указаны у волонтёров в профиле.

Поля таблицы:

– `id` – уникальный идентификатор навыка;

– `name` – название навыка.

3.1.5 Таблица связи волонтёров с навыками

Таблица `volunteer_skills` хранит связь между волонтёрами и их навыками. Она используется для реализации связи many-to-many, так как один

волонтёр может иметь несколько навыков, а один навык может принадлежать разным волонтёрам.

Поля таблицы:

– volunteer_id – внешний ключ на профиль волонтёра;

– skill_id – внешний ключ на навык.

3.1.6 Таблица направлений мероприятий

Таблица directions хранит список направлений волонтёрских мероприятий.

Поля таблицы:

– id – уникальный идентификатор направления;

– name – название направления.

3.1.7 Таблица мероприятий

Таблица events предназначена для хранения волонтёрских мероприятий, создаваемых пользователями. Она содержит сведения о месте проведения, времени, организаторе, статусе, количестве участников и рейтинговых баллах.

Поля таблицы:

– id – уникальный идентификатор мероприятия в формате UUID;

– creator_id – внешний ключ на пользователя, создавшего мероприятие;

– title – название мероприятия;

– description – описание мероприятия;

– country – страна проведения мероприятия;

– city – город проведения;

– location_name – адрес или описание места проведения;

– photo_url – ссылка на фото мероприятия;

– latitude – географическая широта места проведения;

– longitude – географическая долгота места проведения;

– starts_at – дата и время начала мероприятия;

– ends_at – дата и время окончания мероприятия;

– volunteers_needed – требуемое количество волонтёров;

– rating_points – количество рейтинговых баллов за участие;

– status – текущий статус мероприятия;

– present_count – количество участников, отмеченных как присутствующие;

– group_count – количество групп мероприятия;

– reviewed_by – идентификатор пользователя, выполнившего модерацию;

– reviewed_at – дата и время выполнения модерации;

– created_at – дата создания записи;

– updated_at – дата последнего изменения записи.

3.1.8 Таблица связи мероприятий с направлениями

Таблица `events_direction` хранит связь между мероприятиями и направлениями волонтёрской деятельности. Она используется для реализации связи many-to-many.

Поля таблицы:

– `event_id` – внешний ключ на связанное мероприятие;

– `direction_id` – внешний ключ на направление мероприятия.

3.1.9 Таблица заявок на участие

Таблица `event_applications` хранит заявки пользователей на участие в мероприятиях. Она позволяет волонтёру отправить заявку, а организатору принять, отклонить или отменить участие.

Поля таблицы:

– `id` – уникальный идентификатор заявки;

– `event_id` – внешний ключ на мероприятие, на которое отправлена заявка;

– `volunteer_id` – внешний ключ на профиль волонтёра;

– `status` – статус заявки;

– `created_at` – дата создания заявки;

– `updated_at` – дата последнего обновления заявки.

3.1.10 Таблица уведомлений

Таблица `notifications` хранит уведомления, которые отображаются пользователю в мобильном приложении. Уведомления могут быть связаны с мероприятием или заявкой.

Поля таблицы:

– `id` – уникальный идентификатор уведомления;

– `user_id` – внешний ключ на пользователя-получателя;

– `sender_name` – имя отправителя уведомления;

– `event_id` – внешний ключ на связанное мероприятие;

– `application_id` – внешний ключ на связанную заявку;

– `message` – текст уведомления;

– `is_read` – флаг, указывающий, было ли уведомление прочитано;

– `created_at` – дата создания уведомления.

3.1.11 Таблица посещаемости мероприятий

Таблица `event_attendance` хранит сведения о фактическом присутствии участников после начала мероприятия. Она используется организатором для подтверждения явки волонтёров.

Поля таблицы:

– `id` – уникальный идентификатор записи посещаемости;

– `event_id` – внешний ключ на мероприятие;

– `application_id` – внешний ключ на принятую заявку участника;

– is_present – флаг присутствия участника;
– checked_at – дата и время подтверждения присутствия;
– created_at – дата создания записи.

3.1.12 Таблица оценок участников

Таблица event_ratings хранит оценки, выставленные участникам после завершения мероприятия. Эти данные используются для изменения рейтинга профиля.

Поля таблицы:
– id – уникальный идентификатор оценки;
– event_id – внешний ключ на мероприятие;
– rater_id – внешний ключ на профиль пользователя, который выставил оценку;
– rated_id – внешний ключ на профиль оцениваемого участника;
– score – количество начисленных или списанных баллов;
– created_at – дата создания оценки.

3.1.13 Таблица истории рейтинга

Таблица rating_history хранит историю изменения рейтинга профилей. Каждая запись фиксирует изменение баллов и связь с мероприятием, если изменение было выполнено в рамках события.

Поля таблицы:
– id – уникальный идентификатор записи истории;
– profile_id – внешний ключ на профиль пользователя;
– delta – изменение рейтинга;
– event_id – внешний ключ на мероприятие;
– event_title – название мероприятия, связанного с изменением рейтинга;
– created_at – дата создания записи.

3.1.14 Таблица групп мероприятий

Таблица event_groups хранит группы, создаваемые в рамках мероприятия после подтверждения присутствия участников.

Поля таблицы:
– id – уникальный идентификатор группы;
– event_id – внешний ключ на мероприятие;
– group_number – номер группы в рамках мероприятия;
– leader_id – внешний ключ на профиль лидера группы;
– created_at – дата создания группы.

3.1.15 Таблица участников групп

Таблица event_group_members хранит состав групп мероприятия. Она связывает группу с профилями участников и фиксирует роль участника

внутри группы.

Поля таблицы:

– `id` – уникальный идентификатор записи;

– `group_id` – внешний ключ на группу мероприятия;

– `profile_id` – внешний ключ на профиль участника;

– `role` – роль участника в группе;

– `created_at` – дата добавления участника в группу.

3.2 Классы клиентской части

Клиентская часть программного средства реализована в виде мобильного приложения для iOS с использованием фреймворка SwiftUI. В соответствии с архитектурным шаблоном MVVM экранные представления реализованы в виде структур View и отвечают за отображение интерфейса и обработку пользовательских действий, а основная логика экранов вынесена в классы моделей представления ViewModel. Сетевое взаимодействие с серверной частью выполняется через API-сервисы, работающие с REST API, а хранение токенов и управление состоянием авторизованного пользователя обеспечиваются вспомогательными компонентами клиентской части. Поэтому в данном разделе подробно рассматриваются модели представления, API-сервисы, сессия пользователя и компоненты хранения данных.

3.2.1 Класс `AppSession`

Класс `AppSession` предназначен для хранения и управления состоянием пользовательской сессии в клиентском приложении. Он хранит данные авторизации, определяет текущего пользователя и используется при выполнении защищённых запросов к серверной части.

Класс содержит следующие свойства:

– `token: String?` – токен доступа, применяемый при обращении к защищённым методам API;

– `currentUser: UserResponse?` – модель текущего авторизованного пользователя;

– `globalError: String?` – текст глобальной ошибки приложения;

– `isAuthenticated: Bool` – флаг, указывающий, что пользователь вошёл в систему;

– `flow: Flow` – текущее состояние перехода пользователя между экранами авторизации, профиля и основного приложения;

– `isBanned: Bool` – флаг блокировки учётной записи пользователя;

– `authAPI: AuthAPI` – объект для выполнения запросов авторизации.

Класс содержит следующие методы:

– `restoreSession(): Void` – восстанавливает сохранённую пользовательскую сессию;

– `authorize(with: TokenResponse, destination: PostAuthDestination): Void` – сохраняет полученную пару токенов и

обновляет состояние приложения;

– validAccessToken(forceRefresh: Bool): String – возвращает актуальный токен доступа;

– completeProfileSetup(): Void – фиксирует завершение настройки профиля;

– completeEmailVerification(): Void – фиксирует завершение подтверждения электронной почты;

– logout(): Void – завершает сессию пользователя и очищает локальные данные;

– performAuthorizedRequest(operation: (String) async throws -> T): T – выполняет запрос с актуальным токеном и при необходимости обновляет сессию.

3.2.2 Класс AuthAPI

Класс AuthAPI отвечает за взаимодействие клиентской части с серверными маршрутами авторизации. Он формирует HTTP-запросы, кодирует входные данные и декодирует ответы сервера.

Класс содержит следующие методы:

– register(username: String, email: String, password: String): UserResponse – регистрирует нового пользователя;

– login(username: String, password: String): TokenResponse – отправляет данные входа и получает токены доступа;

– refresh(refreshToken: String): TokenResponse – обновляет пару токенов;

– logout(refreshToken: String): Void – завершает активную сессию;

– sendVerificationCode(token: String): Void – запрашивает код подтверждения электронной почты;

– verifyEmail(code: String, token: String): Void – подтверждает электронную почту пользователя;

– forgotPassword(email: String): Void – запускает сценарий восстановления пароля;

– resetPassword(email: String, code: String, newPassword: String): Void – устанавливает новый пароль по коду восстановления.

3.2.3 Класс LoginViewModel

Класс LoginViewModel управляет состоянием экрана входа в приложение. Он хранит введённые пользователем логин и пароль, выполняет проверку формы и обращается к AuthAPI для авторизации.

Класс содержит следующие свойства:

– username: String – имя пользователя, введённое в форме входа;

– password: String – пароль пользователя;

– rememberMe: Bool – флаг, определяющий необходимость сохранения данных пользователя для последующего входа в приложение;

– isLoading: Bool – флаг, указывающий на выполнение запроса авторизации;

– errorMessage: String? – текст ошибки входа.

Класс содержит следующие методы:

– login(): Void – выполняет вход пользователя в приложение;

– clearError(): Void – очищает сообщение об ошибке.

3.2.4 Класс SignUpViewModel

Класс SignUpViewModel управляет состоянием экрана регистрации. Он хранит данные нового пользователя, проверяет корректность электронной почты и пароля, а затем выполняет регистрацию через AuthAPI.

Класс содержит следующие свойства:

– username: String – имя пользователя для регистрации;

– email: String – адрес электронной почты;

– password: String – пароль пользователя;

– confirmPassword: String – повторный ввод пароля;

– isLoading: Bool – флаг выполнения сетевого запроса;

– signUpError: String? – текст ошибки регистрации.

Класс содержит следующие методы:

– signUp(): Void – регистрирует пользователя и переводит его к подтверждению электронной почты;

– clearServerError(): Void – очищает ошибку, полученную от сервера;

– confirmPasswordError(): String? – возвращает ошибку несовпадения паролей;

– passwordValidationErrors(): [String] – возвращает список нарушенных требований к паролю.

3.2.5 Класс KeychainStorage

Класс KeychainStorage отвечает за безопасное локальное хранение токенов авторизации в системном хранилище Keychain. Он используется для сохранения и восстановления пользовательской сессии.

Класс содержит следующие свойства:

– service: String – идентификатор сервиса в Keychain;

– accessAccount: String – ключ записи для access-токена;

– refreshAccount: String – ключ записи для refresh-токена.

Класс содержит следующие методы:

– saveAccessToken(token: String): Void – сохраняет access-токен;

– saveRefreshToken(token: String): Void – сохраняет refresh-токен;

– loadAccessToken(): String? – загружает access-токен;
– loadRefreshToken(): String? – загружает refresh-токен;
– clearAll(): Void – удаляет сохранённые токены.

3.2.6 Класс ProfileAPI

Класс ProfileAPI используется для работы с профилем пользователя. Через него клиент загружает аватар, сохраняет анкету, получает текущий профиль и историю изменения рейтинга.

Класс содержит следующие методы:

– uploadAvatar(data: Data, token: String?): String – загружает изображение профиля и возвращает ссылку на файл;

– saveProfile(requestModel: ProfileUpsertRequest, token: String?): Void – передаёт на сервер данные профиля пользователя;

– getMyProfile(token: String?): ProfileResponse – получает профиль текущего пользователя;

– getRatingHistory(profileID: Int, token: String?): [RatingHistoryItem] – загружает историю начисления или списания рейтинга.

3.2.7 Класс ProfileSetupViewModel

Класс ProfileSetupViewModel управляет экраном создания и редактирования профиля. Он хранит данные волонтёра или организации, выполняет проверку полей и отправляет профиль на сервер через ProfileAPI.

Класс содержит следующие свойства:

– selectedType: ProfileType – выбранный тип профиля: волонтёр или организация;

– avatarImage: UIImage? – изображение профиля, выбранное пользователем;

– firstName: String – имя волонтёра;

– lastName: String – фамилия волонтёра;

– volunteerPhone: String – телефон волонтёра;

– volunteerEmail: String – электронная почта волонтёра;

– selectedSkills: Set<String> – набор выбранных навыков;

– organizationName: String – название организации;

– organizationPhone: String – телефон организации;

– organizationEmail: String – электронная почта организации;

– isLoading: Bool – флаг сохранения профиля;

– errorMessage: String? – текст ошибки.

Класс содержит следующие методы:

– setAvatar(image: UIImage): Void – устанавливает изображение профиля;

– `submit(): Void` – отправляет профиль на сервер;

– `saveProfileChanges(): Void` – сохраняет изменения существующего профиля;

– `loadMyProfile(): Void` – загружает профиль текущего пользователя;

– `makeProfileSnapshot(): ProfileSnapshot` – создаёт снимок текущего состояния формы;

– `restoreProfileSnapshot(snapshot: ProfileSnapshot): Void` – восстанавливает форму из сохранённого снимка.

3.2.8 Класс `EventAPI`

Класс `EventAPI` отвечает за базовые операции с мероприятиями в клиентской части. Он используется для создания события, получения карточки мероприятия, загрузки списков и изменения основных статусов жизненного цикла.

Класс содержит следующие методы:

– `createEvent(request: CreateEventRequest, token: String?): EventResponse` – создаёт новое мероприятие на сервере;

– `getEvent(id: String, token: String?): EventResponse` – получает подробную информацию о мероприятии;

– `getMyEvents(filter: MyEventsFilter?, token: String?): [EventResponse]` – получает список мероприятий текущего пользователя;

– `getEventFeed(searchText: String?, filters: EventFeedFilters, token: String?): [EventResponse]` – получает список доступных мероприятий с учётом поиска и фильтров;

– `deleteEvent(id: String, token: String): Void` – удаляет мероприятие;

– `startEvent(id: String, token: String): EventResponse` – переводит мероприятие в статус проведения;

– `cancelEvent(id: String, reason: String, token: String): EventResponse` – отменяет мероприятие с указанием причины;

– `finishEvent(id: String, token: String): EventResponse` – завершает мероприятие.

3.2.9 Класс `EventParticipantsAPI`

Класс `EventParticipantsAPI` отвечает за клиентские запросы, связанные с заявками, участниками, посещаемостью и рейтинговыми оценками. Выделение этого класса позволяет отделить работу с участниками от общих операций с мероприятиями.

Класс содержит следующие методы:

– `applyToEvent(id: String, token: String): EventResponse` – отправляет заявку пользователя на участие;

– cancelApplication(eventID: String, token: String): EventResponse – отменяет заявку текущего пользователя;

– acceptApplication(id: Int, token: String): EventResponse – принимает заявку волонтёра;

– rejectApplication(id: Int, token: String): EventResponse – отклоняет заявку волонтёра;

– removeParticipant(applicationID: Int, reason: String, token: String): EventResponse – исключает участника из мероприятия;

– getEventApplications(eventID: String, token: String): [EventParticipantResponse] – получает список заявок и участников мероприятия;

– getAttendance(eventID: String, token: String): [AttendanceItem] – получает список участников для подтверждения присутствия;

– confirmAttendance(eventID: String, presentApplicationIDs: [Int], token: String): EventResponse – передаёт список присутствующих участников;

– getRatableProfiles(eventID: String, token: String): [RatableProfile] – получает список участников, доступных для оценки;

– submitRatings(eventID: String, ratings: [RatingItem], token: String): EventResponse – отправляет рейтинговые оценки участников.

3.2.10 Класс `EventGroupChatAPI`

Класс EventGroupChatAPI отвечает за клиентские операции группировки и чатов мероприятия. Он используется после подтверждения присутствия, когда организатор распределяет участников по группам и участники обмениваются сообщениями.

Класс содержит следующие методы:

– saveGroupCountDraft(eventID: String, groupCount: Int, token: String): EventResponse – сохраняет выбранное количество групп без окончательного подтверждения;

– confirmGrouping(eventID: String, groupCount: Int, token: String): EventResponse – подтверждает переход мероприятия к группировке;

– confirmGroups(eventID: String, token: String): EventResponse – подтверждает итоговое распределение участников;

– getGroups(eventID: String, token: String): [EventGroup] – получает список групп мероприятия;

– addGroup(eventID: String, token: String): EventGroup – создаёт новую группу;

– deleteGroup(eventID: String, groupNumber: Int,

token: String): Void – удаляет выбранную группу;

– autoDistribute(eventID: String, token: String): [EventGroup] – автоматически распределяет участников по группам;

– setGroupLeader(eventID: String, groupNumber: Int, profileID: Int, token: String): EventGroup – назначает лидера группы;

– removeGroupLeader(eventID: String, groupNumber: Int, token: String): EventGroup – снимает лидера группы;

– addGroupMember(eventID: String, groupNumber: Int, profileID: Int, token: String): EventGroup – добавляет участника в группу;

– removeGroupMember(eventID: String, groupNumber: Int, profileID: Int, token: String): EventGroup – удаляет участника из группы;

– fetchMessages(eventID: String, token: String): [ChatMessage] – получает сообщения общего чата мероприятия;

– sendMessage(eventID: String, content: String?, photoURL: String?, token: String): ChatMessage – отправляет сообщение в общий чат мероприятия;

– getChats(eventID: String, token: String): [ChatRoom] – получает список доступных чатов мероприятия;

– getChatMessages(eventID: String, chatID: Int, token: String): [ChatMessage] – получает сообщения выбранного чата;

– getChatParticipants(eventID: String, chatID: Int, token: String): [EventParticipantResponse] – получает участников выбранного чата;

– sendChatMessage(eventID: String, chatID: Int, content: String?, photoURL: String?, token: String): ChatMessage – отправляет сообщение в выбранный чат.

3.2.11 Класс NotificationAPI

Класс NotificationAPI отвечает за получение и изменение уведомлений пользователя. Он применяется на экране списка уведомлений и при переходе пользователя к связанному событию или заявке.

Класс содержит следующие методы:

– fetchNotifications(token: String): [AppNotificationItem] – получает список уведомлений текущего пользователя;

– markNotificationRead(id: Int, token: String): Void – помечает выбранное уведомление как прочитанное;

– clearNotifications(token: String): Void – удаляет или очищает уведомления пользователя.

3.2.12 Класс `GeocodingAPI`

Класс `GeocodingAPI` обеспечивает взаимодействие с серверными методами геокодирования. Он используется при поиске адреса и при выборе точки мероприятия на карте.

Класс содержит следующие методы:

– `forwardGeocode(query: String, context: EventLocationContext): [GeocodingSuggestion]` – получает варианты адресов по строке поиска;

– `reverseGeocode(coordinate: CLLocationCoordinate2D, context: EventLocationContext): EventLocationSelection?` – определяет адрес по координатам выбранной точки;

– `geocodeCity(city: String, country: String, area: CountrySearchArea): CLLocationCoordinate2D?` – получает координаты города для начального позиционирования карты.

3.2.13 Класс `EventDetailsViewModel`

Класс `EventDetailsViewModel` представляет логическую модель состояния экрана подробной информации о мероприятии. Он отвечает за загрузку мероприятия, работу с заявкой пользователя и получение списка участников.

Класс содержит следующие свойства:

– `event: EventResponse` – текущее мероприятие;

– `applications: [EventApplicationResponse]` – список заявок на участие;

– `participants: [EventParticipantResponse]` – список участников мероприятия;

– `isLoading: Bool` – флаг загрузки данных;

– `errorMessage: String?` – текст ошибки.

Класс содержит следующие методы:

– `loadEvent(eventId: UUID, session: AppSession): Void` – загружает данные мероприятия;

– `applyToEvent(eventId: UUID, session: AppSession): Void` – отправляет заявку на участие;

– `loadParticipants(eventId: UUID, session: AppSession): Void` – загружает список участников;

– `deleteEvent(eventId: UUID, session: AppSession): Void` – удаляет мероприятие.

3.2.14 Класс `MyEventsViewModel`

Класс `MyEventsViewModel` управляет экраном мероприятий текущего пользователя. Он загружает список созданных мероприятий и событий, в которых пользователь участвует.

Класс содержит следующие свойства:

– events: [EventResponse] – список мероприятий пользователя;

– isLoading: Bool – флаг загрузки данных;

– errorMessage: String? – текст ошибки.

Класс содержит следующие методы:

– loadMyEvents(filter: MyEventsFilter?): Void – загружает мероприятия пользователя с учётом фильтра;

– prependOrUpdate(event: EventResponse): Void – добавляет новое мероприятие в список или обновляет существующее.

3.2.15 Класс NotificationViewModel

Класс NotificationViewModel управляет экраном уведомлений. Он загружает уведомления пользователя, помечает их как прочитанные и очищает список.

Класс содержит следующие свойства:

– notifications: [AppNotificationItem] – список уведомлений пользователя;

– isLoading: Bool – флаг загрузки уведомлений;

– errorMessage: String? – текст ошибки;

– hasNotifications: Bool – флаг наличия непрочитанных уведомлений.

Класс содержит следующие методы:

– loadNotifications(): Void – загружает уведомления пользователя;

– markAsRead(notificationId: Int): Void – помечает уведомление как прочитанное;

– clearAll(): Void – очищает список уведомлений.

3.2.16 Класс EventViewModel

Класс EventViewModel управляет формой создания мероприятия. Он хранит введённые пользователем данные, проверяет корректность формы, подготавливает модель CreateEventRequest и вызывает EventAPI.

Класс содержит следующие свойства:

– eventTitle: String – название создаваемого мероприятия;

– selectedDirection: String – выбранное направление волонтёрской деятельности;

– eventDescription: String – описание мероприятия;

– selectedCoordinate: CLLocationCoordinate2D? – координаты выбранного места проведения;

– startDate: Date? – дата начала мероприятия;

– volunteersCount: Int – требуемое количество волонтёров;

– ratingPoints: Int – количество рейтинговых баллов за участие;

– canSubmit: Bool – флаг готовности формы к отправке.

Класс содержит следующие методы:

– loadProfileContextIfNeeded(): Void – загружает данные профиля и контекст местоположения организатора;

– setLocation(selection: EventLocationSelection): Void – сохраняет выбранный адрес и координаты;

– setEventPhoto(image: UIImage): Void – подготавливает фотографию мероприятия к загрузке;

– submit(): EventResponse? – загружает фотографию при необходимости и отправляет запрос создания события;

– resetFormKeepingContext(): Void – очищает форму после успешного создания мероприятия.

3.2.17 Класс **EventLocationPickerViewModel**

Класс EventLocationPickerViewModel управляет экраном выбора местоположения. Он выполняет поиск адресов, обратное геокодирование и обновляет выбранную точку на карте.

Класс содержит следующие свойства:

– query: String – текст поискового запроса;

– suggestions: [GeocodingSuggestion] – список найденных адресов;

– selectedLocation: EventLocationSelection? – выбранный пользователем адрес;

– cameraCoordinate: CLLocationCoordinate2D – координата центра карты;

– isSearching: Bool – флаг выполнения поискового запроса.

Класс содержит следующие методы:

– loadInitialLocationIfNeeded(): Void – устанавливает начальную позицию карты;

– chooseSuggestion(suggestion: GeocodingSuggestion): Void – выбирает адрес из списка подсказок;

– handleMapTap(coordinate: CLLocationCoordinate2D): Void – обрабатывает нажатие пользователя на карту;

– handleMapRegionDidChange(coordinate: CLLocationCoordinate2D): Void – запускает обратное геокодирование после изменения области карты;

– clearSearch(): Void – очищает поисковую строку и список подсказок.

3.2.18 Класс **EventParticipantsManagementViewModel**

Класс EventParticipantsManagementViewModel отвечает за управление заявками и участниками мероприятия. Он используется организатором для принятия заявок, удаления участников и обновления списка волонтёров.

Класс содержит следующие свойства:

– eventID: String – идентификатор мероприятия, для которого выполняется управление;

– applications: [EventParticipantResponse] – общий список заявок и участников мероприятия;

– participants: [EventParticipantResponse] – вычисляемый список принятых участников;

– applicationRequests: [EventParticipantResponse] – вычисляемый список заявок, ожидающих решения;

– isLoading: Bool – флаг загрузки данных;

– errorMessage: String? – текст ошибки, отображаемый в интерфейсе.

Класс содержит следующие методы:

– loadApplications(): Void – загружает актуальный список заявок и участников;

– acceptApplication(application: EventParticipantResponse): Void – принимает выбранную заявку;

– rejectApplication(application: EventParticipantResponse): Void – отклоняет выбранную заявку;

– removeParticipant(participant: EventParticipantResponse, reason: String): Void – исключает участника из мероприятия с указанием причины;

– reviewApplication(application: EventParticipantResponse, accept: Bool): Void – выполняет общий сценарий принятия или отклонения заявки.

3.2.19 Класс **EventRatingViewModel**

Класс EventRatingViewModel представляет логическую модель состояния экрана оценки участников. Он хранит список профилей для оценки, выбранные баллы и состояние отправки результата.

Класс содержит следующие свойства:

– eventID: String – идентификатор мероприятия;

– isOrganizerVerified: Bool – флаг подтверждённой организации;

– eventRatingPoints: Int – количество баллов мероприятия;

– profiles: [RatableProfile] – список участников, доступных для оценки;

– selectedScores: [Int: Int?] – выбранные оценки по идентификаторам профилей;

– isLoading: Bool – флаг, указывающий на выполнение загрузки профилей;

– isSubmitting: Bool – флаг, указывающий на выполнение отправки оценок;

– errorMessage: String? – текст ошибки.

Класс содержит следующие методы:

– `loadProfiles(): Void` – загружает список участников для оценки;

– `submitRatings(): Void` – отправляет выбранные рейтинговые оценки на сервер.

3.3 Классы серверной части

Серверная часть программного средства реализована на основе REST API и отвечает за обработку запросов мобильного приложения, авторизацию пользователей, хранение данных, выполнение бизнес-логики, модерацию мероприятий и обмен информацией между участниками системы. В серверной части используются модели базы данных, схемы передачи данных и сервисные классы, которые обеспечивают работу с пользователями, профилями, мероприятиями, заявками, уведомлениями, геокодированием, рейтингом, группами, чатами и административным Telegram-ботом.

3.3.1 Класс `AuthService`

Класс `AuthService` представляет сервис авторизации серверной части. Он выполняет нормализацию логина, создание администратора по умолчанию, выдачу пары токенов и отзыв refresh-токена.

Класс содержит следующие методы:

– `normalize_username(username: str): String` – приводит имя пользователя к единому формату;

– `create_default_admin(db: Session): Void` – создаёт администратора по умолчанию при первом запуске сервера;

– `issue_token_pair(user: User, db: Session): TokenResponse` – создаёт пару access- и refresh-токенов;

– `revoke_refresh_token(raw_refresh_token: String, db: Session): RefreshToken` – отзывает refresh-токен пользователя.

3.3.2 Класс `ProfileService`

Класс `ProfileService` представляет серверную логику работы с профилем пользователя. Он отвечает за получение текущего профиля, сохранение данных профиля и загрузку истории рейтинга.

Класс содержит следующие методы:

– `get_my_profile(db: Session, user: User): Profile` – получает профиль текущего пользователя;

– `upsert_profile(db: Session, current_user: User, request: ProfileUpsertRequest): Profile` – создаёт или обновляет профиль пользователя;

– `get_rating_history(db: Session, current_user: User): [RatingHistoryItemResponse]` – получает историю изменения рейтинга профиля.

3.3.3 Класс `NotificationService`

Класс `NotificationService` реализует серверную работу с уведомлениями. Он создаёт уведомления, возвращает их список для пользователя, помечает уведомления как прочитанные и очищает список.

Класс содержит следующие методы:

– `create(userId: int, message: str): Notification` – создаёт новое уведомление для пользователя;

– `list_for_user(user: User): [Notification]` – возвращает список уведомлений пользователя;

– `mark_read_for_user(notificationId: int, user: User): Notification` – помечает уведомление как прочитанное;

– `clear_for_user(user: User): Void` – удаляет уведомления пользователя.

3.3.4 Класс `EmailService`

Класс `EmailService` представляет серверный сервис отправки писем. Он используется для отправки кодов подтверждения электронной почты и кодов восстановления пароля.

Класс содержит следующие методы:

– `send_verification_code(to_email: str, code: str): Void` – отправляет код подтверждения электронной почты;

– `send_password_reset_code(to_email: str, code: str): Void` – отправляет код восстановления пароля.

3.3.5 Класс `EventService`

Класс `EventService` реализует серверную логику, связанную с основным жизненным циклом мероприятия. Он отвечает за создание, модерацию, получение списков событий, запуск, отмену и завершение мероприятия.

Класс содержит следующие методы:

– `create_event(db: Session, payload: CreateEventRequest, user: User, background_tasks: BackgroundTasks?): EventResponse` – создаёт мероприятие и отправляет его на модерацию;

– `approve_event(db: Session, event_id: str, admin: User): EventResponse` – подтверждает мероприятие администратором;

– `get_event(db: Session, event_id: str): Event?` – получает объект мероприятия по идентификатору;

– `get_event_response(db: Session, event_id: str, user: User?): EventResponse?` – возвращает мероприятие в формате ответа API;

– `list_my_events(db: Session, user: User, event_filter: str?): [EventResponse]` – получает список

мероприятий текущего пользователя;

– `list_events(db: Session, country: str?, city: str?): [CurrentCountryEventResponse]` – получает мероприятия для отображения на карте;

– `list_event_feed(db: Session, search_query: str?, direction: str?, when: str?, country: str?, city: str?): [EventResponse]` – возвращает ленту мероприятий с поиском и фильтрами;

– `list_events_for_current_user_country(db: Session, user: User): [CurrentCountryEventResponse]` – получает мероприятия для страны текущего пользователя;

– `cancel_event(db: Session, event_id: str, user: User, reason: str): EventResponse` – отменяет мероприятие с указанием причины;

– `delete_event(db: Session, event_id: str, user: User): Void` – удаляет мероприятие;

– `start_event(db: Session, event_id: str, user: User): EventResponse` – переводит мероприятие в статус проведения;

– `finish_event(db: Session, event_id: str, user: User): EventResponse` – завершает мероприятие.

3.3.6 Класс `EventParticipantsService`

Класс `EventParticipantsService` выделяет серверную логику, связанную с заявками, участниками, подтверждением присутствия и начислением рейтинга. Он обрабатывает действия волонтёров и организатора после создания мероприятия.

Класс содержит следующие методы:

– `apply_to_event(db: Session, event_id: str, user: User): EventResponse` – создаёт заявку волонтёра на участие;

– `cancel_application(db: Session, event_id: str, user: User): EventResponse` – отменяет заявку текущего пользователя;

– `accept_application(db: Session, application_id: int, user: User): EventResponse` – принимает заявку участника;

– `reject_application(db: Session, application_id: int, user: User): EventResponse` – отклоняет заявку участника;

– `list_event_applications(db: Session, event_id: str, user: User): [EventParticipantResponse]` – возвращает список заявок и участников мероприятия;

– `remove_participant(db: Session, application_id: int, user: User, reason: str): EventResponse` – исключает участника из мероприятия;

– `get_attendance(db: Session, event_id: str, user: User): [AttendanceItemResponse]` – получает данные для отметки

присутствия;

– confirm_attendance(db: Session, event_id: str, user: User, present_application_ids: [int]): EventResponse – подтверждает список присутствующих участников;

– get_ratable_profiles(db: Session, event_id: str, user: User): [RatableProfileResponse] – получает список участников для оценки;

– submit_ratings(db: Session, event_id: str, user: User, ratings: [RatingItem]): EventResponse – сохраняет оценки и изменяет рейтинг участников.

3.3.7 Класс `EventGroupChatService`

Класс EventGroupChatService объединяет серверную логику группировки и чатов мероприятия. Он используется после подтверждения присутствия участников и обеспечивает создание групп, назначение лидеров и обмен сообщениями.

Класс содержит следующие свойства:

– LEADERS_CHAT_ID: int – служебный идентификатор чата лидеров;

– _GROUP_EDIT_STATUSES: set[str] – набор статусов, в которых разрешено редактирование групп.

Класс содержит следующие методы:

– save_group_count_draft(db: Session, event_id: str, user: User, group_count: int): EventResponse – сохраняет черновое количество групп;

– confirm_grouping(db: Session, event_id: str, user: User, group_count: int): EventResponse – подтверждает переход к этапу группировки;

– confirm_groups(db: Session, event_id: str, user: User): EventResponse – подтверждает итоговую группировку;

– get_groups(db: Session, event_id: str, user: User): [GroupResponse] – получает список групп мероприятия;

– add_group(db: Session, event_id: str, user: User): GroupResponse – создаёт новую группу;

– delete_group(db: Session, event_id: str, group_number: int, user: User): Void – удаляет группу;

– auto_distribute(db: Session, event_id: str, user: User): [GroupResponse] – автоматически распределяет участников по группам;

– set_group_leader(db: Session, event_id: str, group_number: int, profile_id: int, user: User): GroupResponse – назначает лидера группы;

– remove_group_leader(db: Session, event_id: str,

group_number: int, user: User): GroupResponse – снимает лидера группы;

– add_group_member(db: Session, event_id: str, group_number: int, profile_id: int, user: User): GroupResponse – добавляет участника в группу;

– remove_group_member(db: Session, event_id: str, group_number: int, profile_id: int, user: User): GroupResponse – удаляет участника из группы;

– get_messages(db: Session, event_id: str, user: User): [MessageResponse] – получает сообщения общего чата мероприятия;

– send_message(db: Session, event_id: str, user: User, content: str?, photo_url: str?): MessageResponse – отправляет сообщение в общий чат;

– get_chats(db: Session, event_id: str, user: User): [ChatRoomResponse] – получает доступные чаты мероприятия;

– get_chat_messages(db: Session, event_id: str, chat_id: int, user: User): [MessageResponse] – получает сообщения выбранного чата;

– get_chat_participants(db: Session, event_id: str, chat_id: int, user: User): [EventParticipantResponse] – получает участников выбранного чата;

– send_chat_message(db: Session, event_id: str, chat_id: int, user: User, content: str?, photo_url: str?): MessageResponse – отправляет сообщение в выбранный чат.

3.3.8 Класс GeocodingService

Класс GeocodingService реализует серверную работу с геокодированием. Он принимает запросы от мобильного приложения, обращается к внешнему геосервису и возвращает нормализованные адреса и координаты.

Класс содержит следующие свойства:

– YANDEX_GEOCODER_URL: str – адрес внешнего геокодирующего сервиса;

– YANDEX_GEOCODER_API_KEY: str – ключ доступа к внешнему API;

– forward_cache: TTLCacheStore – кэш прямого геокодирования;

– reverse_cache: TTLCacheStore – кэш обратного геокодирования.

Класс содержит следующие методы:

– forward_geocode(db: Session, user: User | None, query: str): ForwardGeocodeResponse – выполняет поиск

координат по адресу;

– `reverse_geocode(db: Session, user: User | None, latitude: float, longitude: float): ReverseGeocodeResponse` – получает адрес по координатам;

– `_resolve_user_location(db: Session, user: User | None): LocationContext` – определяет географический контекст пользователя;

– `_call_yandex_forward(query: str): list[GeocodingSuggestionResponse]` – обращается к внешнему сервису прямого геокодирования.

3.3.9 Класс `TelegramAdminBot`

Класс `TelegramAdminBot` используется для администрирования и модерации мероприятий через Telegram. Он отправляет администраторам уведомления о новых событиях и обрабатывает команды подтверждения, отклонения и управления пользователями.

Класс содержит следующие свойства:

– `token: str` – токен Telegram-бота;

– `admin_ids: set[int]` – список идентификаторов администраторов;

– `_offset: int` – смещение для получения новых обновлений Telegram;

– `_running: bool` – флаг, указывающий, активен ли polling-цикл;

– `_pending_reject_reasons: dict[int, str]` – временное хранилище причин отклонения мероприятий;

– `_pending_reject_confirmations: dict[int, dict[str, str]]` – временное хранилище подтверждений отклонения мероприятий;

– `_event_locks: dict[str, tuple[int, float]]` – блокировки мероприятий при одновременной обработке администраторами.

Класс содержит следующие методы:

– `is_configured(): bool` – возвращает флаг наличия настроек Telegram-бота;

– `run_polling(): None` – запускает получение сообщений и команд Telegram-бота;

– `stop(): None` – останавливает работу Telegram-бота;

– `notify_admins_about_event(db: Session, event: Event, creator: User): None` – отправляет администраторам сведения о новом мероприятии;

– `notify_admins_about_event_id(event_id: str): None` – отправляет администраторам сведения о новом мероприятии;

– `_handle_message(message: dict): None` – обрабатывает текстовые команды администратора;

– `_handle_callback(callback: dict): None` – обрабатывает

нажатия кнопок Telegram;

– `_approve_event(callback_id: str | None, event_id: str): None` – подтверждает мероприятие по команде администратора;

– `_reject_event(event_id: str, reason: str, admin_telegram_id: int, callback_id: str?): None` – отклоняет мероприятие с указанной причиной;

– `_confirm_reject(callback_id: str?, admin_telegram_id: int, event_id: str): None` – подтверждает отклонение мероприятия после ввода причины;

– `_list_profiles(search_query: str): str` – формирует список профилей для администратора;

– `_list_organizations(search_query: str): str` – формирует список организаций;

– `_verify_organization(arg: str): str` – подтверждает профиль организации;

– `_ban_profile(arg: str): str` – блокирует выбранный аккаунт пользователя;

– `_unban_profile(arg: str): str` – снимает блокировку с аккаунта пользователя;

– `_freeze_profile(arg: str): str` – временно замораживает аккаунт пользователя;

– `_api(method: str, payload: dict): dict` – выполняет запрос к Telegram Bot API.

3.4 Выводы по разделу

В результате функционального проектирования были выделены основные классы клиентской и серверной частей программного средства. Классы клиентской части предназначены для формирования пользовательского интерфейса, хранения состояния экранов, обработки действий пользователя и выполнения сетевого взаимодействия с сервером. Классы серверной части обеспечивают описание сущностей предметной области, обработку заявок на участие, управление мероприятиями, формирование групп участников, отправку уведомлений и выполнение рейтинговых операций.

Таким образом, разработанная структура классов отражает логику функционирования приложения и демонстрирует взаимосвязь между мобильным клиентом, серверной частью и данными, хранящимися в базе данных.

Диаграмма последовательности взаимодействия блоков между собой приведена на чертеже ГУИР.400201.311 РР.2.